Ders 4 Genişletilmiş Notu: Yapısal Kalıplar: Adapter, Bridge ve Composite

Ders 4 Genişletilmiş Notu: Yapısal Kalıplar: Adapter, Bridge ve Composite

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yapısal Kalıplara Giriş
- 2. Adapter Kalıbı
- 3. Bridge Kalıbı: Amaç ve GoF Örneği
- 4. Bridge'in Yapısı ve Kullanım Durumları
- 5. Bridge Uygulama Örneği: İki Çizim Kütüphanesi
- 6. Tartışma: Shape'in Sorumlulukları ve Adapter'lı Alternatif
- 7. Tartışma: Varlık Sınıflarına Davranış Yüklemek
- 8. Composite Kalıbı: Amaç ve Örnek
- 9. Composite'in Yapısı ve Kullanım Durumları
- 10. Composite Gerçekleme Ayrıntıları ve Zayıf Yönleri
- 11. Uygulama Örneği: Makine Üretim Kataloğu
- 12. Kod Kusuru: Tekrarlanan Kod ve ReportV2
- Örnek Sorular

1. Yapısal Kalıplara Giriş

Önceki derste yaratımsal (creational) kalıplar Singleton ile tamamlanmıştı. Bu derste Dörtlü Çete'nin (Gang of Four, GoF) kataloğundaki yapısal (structural) kalıplara geçildi. Yapısal kalıplar sınıfların ve nesnelerin daha büyük yapılar oluşturmak üzere nasıl bir araya getirileceğiyle ilgilenir. Derste sırasıyla Adapter, Bridge ve Composite kalıpları işlendi; Bridge ve Composite için birer uygulama örneği kodlarıyla incelendi, Bridge örneği üzerine de uzun bir öğrenci tartışması yapıldı.

2. Adapter Kalıbı

2.1 Amaç

Elimizde bir istekçi (client) sınıf ve bir hizmet veren (sunucu) sınıf vardır. İstekçi, hizmet sınıfının belli bir metodunu belli bir adla çağırmak istiyor. Fakat hizmeti verecek sınıfta o işi yapan metot başka bir adla (başka bir imzayla) tanımlanmış. Üstelik iki sınıfın da kaynak kodunu değiştiremiyoruz.

Kalıbın önerdiği çözüm şudur: araya, istekçinin beklediği metot imzasına sahip bir sınıf konur. Bu sınıfa istekçinin çağırmak istediği metot eklenir ve bu metodun gövdesinde asıl işi yapacak sınıfın ilgili metodu çağrılır. Çoğu durumda bu, tek satırlık bir koddur.

Hoca kalıbın adını günlük hayattan bir benzetmeyle açıkladı. Farklı ülkelerin elektrik fişi ve priz standartları farklıdır. Örneğin İngiltere'de prizlerde faz ve nötr deliklerinin yanında üstte üçüncü bir delik daha bulunur. Bu delik güvenlik içindir: fişin üstteki daha uzun, dörtgen bacağı önce girer ve içerideki küçük dili aşağı iterek faz ve nötr deliklerini kapatan kapakları

açar. Bizim iki bacaklı fişlerimizi orada kullanmak için marketlerde bile satılan "travel adapter" setleri kullanılır: adaptörün bir yüzünde İngiliz standardında üç erkek bacak, diğer yüzünde iki bacaklı fişin girebileceği yuvalar vardır. Cihaz önce adaptöre, adaptör de duvardaki prize takılır. Kalıp da aynı adı taşır ve aynı işi yapar.

2.2 Kalıp yapısı

Hoca kalıp yapısını şöyle anlattı: client, `Target` adlı bir arayüzün (ya da soyut sınıfın) `request()` metodunu çağırmak istiyor. Client'ın kaynak kodu elimizde değil, bu yüzden onun `request()` çağırmasını değiştiremeyiz. Elimizde aynı işi yapan `Adaptee` adlı bir sınıf var; onun metodu `specificRequest()` adını taşıyor. Çözüm olarak `Target`'ı gerçekleyen (ya da `Target` soyut sınıfının alt sınıfı olan) bir `Adapter` sınıfı yazılır; onun `request()` metodu yalnız elimizdeki hazır sınıfın `specificRequest()` metodunu çağırır.

Kalıp şu koşulda kullanılır: elimizdeki sınıfın istekçi tarafından kullanılmasını önleyen tek engel, istekçinin metot çağrılarıyla elimizdeki sınıfın anladığı metot imzalarının adlarının farklı olmasıdır.

> **Ek bilgi (derste anlatılmadı):** GoF kitabı Adapter'ın iki biçimini ayırır. "Sınıf adaptörü" çoklu kalıtımla hem `Target`'tan hem `Adaptee`'den türer; "nesne adaptörü" ise `Adaptee` nesnesini bir üye alanda tutup çağrıları ona iletir. Java'da sınıflar arasında çoklu kalıtım olmadığı için derste gösterilen örnek nesne adaptörü biçimindedir.

2.3 Örnek: çizim programına hazır bir daire sınıfı eklemek

Bir çizim programı üzerinde çalışılıyor ve programın bir kısmı halihazırda gerçeklenmiş (ekrandan, 00:10:00):

```
Shape (soyut)
  +setLocation()  +getLocation()  +display()  +fill()  +setColor()  +undisplay()
   △── Point   {+display(), +fill(), +undisplay()}
   △── Line    {+display(), +fill(), +undisplay()}
   △── Square  {+display(), +fill(), +undisplay()}

XXCircle   (Shape ile ilişkisi: ???)
  +setLocation()  +getLocation()  +displayIt()  +fillIt()  +setItsColor()  +undisplayIt()
```

`Shape` soyut bir sınıftır; bazı metotları soyuttur (slaytta italik yazılanlar). Nokta, çizgi ve kare kodlanmış. Sırada daire var. Hoca daire kodunu yazmanın kolay olmadığını anlattı: ekran nihayetinde kare piksellerden oluşan iki boyutlu bir matristir; buradan yumuşak, yuvarlak bir kenar çizmek, tam daire çizilemeyeceği için daireye nasıl yaklaşılacağını bulmak, hatta kenar yumuşatma (anti-aliasing) yapmak için farklı algoritmalar vardır. Bu işi sıfırdan yazmaya üşenen programcı, başka bir projesinde güzelce kodlanmış bir `XXCircle` sınıfı olduğunu hatırlar. Belki o projenin kaynak kodu bile kaybolmuştur, yalnız `.class` dosyaları bulunabilmiştir; ya da kaynak kod vardır ama kopyalayıp değiştirmek istenmez. Konum ayarlama, gösterme, doldurma, rengini değiştirme ve ekrandan silme işlerinin hepsi bu eski kodda yapılmıştır; yalnız metot adları farklıdır: `display` yerine `displayIt`, `fill` yerine `fillIt` gibi. Bu yüzden `XXCircle` doğrudan `Shape` hiyerarşisine eklenemez.

Çözüm (ekrandan, 00:11:00): `Shape`'ten kalıtımla bir `Circle` sınıfı oluşturulur ve üstteki soyut metotların somut gövdeleri yazılır. Ama gövdeler işi kendisi yapmaz, `XXCircle`'a iletir:

```
class Circle extends Shape {
    ...
    private XXCircle myXXCircle;
    ...
    public Circle() {
        myXXCircle = new XXCircle();
    }

    public void display() {
        myXXCircle.displayIt();
    }
```

```
    ...
}
```

> **Ders sonrası not:** Slayttaki kodda metot başlığı `void public display()` olarak yazılmıştır. Java'da erişim belirleyicisi dönüş türünden önce gelir; doğrusu `public void display()` biçimindedir. Yukarıdaki kodda bu düzeltilmiştir.

Slayttaki şemada `Circle` ile `XXCircle` arasında dolu elmaslı bir parça-bütün ilişkisi çizilmiştir. Hoca bu örnekte ilişkinin aslında birebir bir sahiplik olduğunu, elmasa kesinlikle gerek olmadığını, düz çizgili tek yönlü bir sahiplik okunun yeterli olduğunu söyledi. Ancak arada bir kalıp gerçeklemesi olduğunu vurgulamak için ilişkiyi basit bir sahiplikten öte göstermek isteyen elmas notasyonunu isteğe bağlı olarak kullanabilir. Özetle `Circle` sınıfının kodu, bütün işlevleriyle sıfırdan yazılmaz; işlevler elimizdeki sınıfa iletilir, delege edilir, yönlendirilir.

2.4 Soru: Başka uyumsuz sınıflar da olursa?

Soru: Örnekte farklı bir daire türü (`XXCircle`) vardı ve onu bir adaptör sınıfıyla bağladık. Bir tane daha farklı türümüz olursa ikinci bir adaptör sınıfı mı yazmamız gerekir?

Cevap: Evet. Örneğin ileride elips, beşgen ya da altıgen sınıfları da eklenecek ve `XXPentagon` gibi daha önce kodlanmış bir sınıf varsa, `Circle` ve `XXCircle`'da yapıldığı gibi onun için de ayrı bir adaptör sınıfı kodlanır.

3. Bridge Kalıbı: Amaç ve GoF Örneği

3.1 Amaç

Bridge kalıbının amacı, bir nesnenin arayüzüyle (soyutlamasıyla) gerçeklemesini, yani kodlanmasını birbirinden ayırmaktır. Hoca ilk okumada bunun tuhaf gelebileceğini söyledi: nesne zaten üye alanları ve üye metotlarıyla bir bütündür; "arayüz" denince de pek çok dilde bir `interface` tanımlayıp bir sınıfın onu gerçeklemesi akla gelir. Ama burada kastedilen bundan daha ayrıntılı bir şeydir. Windows programlaması yapanlar bu kalıbı "Handle" adıyla da bilirler.

3.2 GoF örneği: farklı pencere ortamları

GoF'un verdiği örnek bir GUI çerçevesidir (framework). Bir pencere soyutlaması kodlanacak, ama program farklı masaüstü ortamlarını (hocanın örnekleriyle GNOME, KDE vb.) desteklemelidir. İlk akla gelen yol, ortak işleri yapan soyut bir `Window` sınıfı yazıp `GnomeWindow`, `KdeWindow` gibi alt sınıflar türetmektir.

Sorun, yeni bir pencere türü eklenmek istendiğinde ortaya çıkar. Örneğin normal pencereye ek olarak şeffaf bir pencere ya da küçültüldüğünde görev çubuğuna değil sistem çubuğuna (system tray) inen ve orada kalan bir `IconWindow` yazılmak istensin. `Window`'dan kalıtımla tek bir `IconWindow` sınıfı türetmek yetmez; desteklenen her pencere yönetim sistemi için `XIconWindow`, `PMIconWindow`, `GnomeIconWindow`, `KdeIconWindow` gibi ayrı alt sınıflar gerekir. İki ortam varsa tek bir yeni işlev için iki yeni sınıf, üç-dört ortam varsa üç-dört yeni sınıf eklenir. Hoca "böyle gidersek yolun sonu aydınlık değil" dedi.

Hoca bu durumu Abstract Factory'den de ayırdı: burada bir ürünler ailesi yoktur; tek bir pencere soyutlaması vardır ve bu soyutlama farklı ortamlara özelleştirilmek istenmektedir.

3.3 Çözüm

Pencerenin bir soyutlaması vardır (örneğin dört çizgiyle bir dikdörtgen çizilir, içindeki alan penceredir) ve bu soyutlamanın belli pencere yöneticilerinde gerçeklenmesi vardır. Bridge, soyutlama ile gerçeklemeleri tek bir kalıtım hiyerarşisinde toplamak yerine iki ayrı hiyerarşiye koyar: birincisi pencere soyutlaması ve türevleri, ikincisi pencere gerçeklemesi ve türevleri. İki hiyerarşi arasında bir sahiplik ilişkisi kurulur; kalıba adını veren "köprü" bu ilişkidir (ekrandan, 00:20:00):

```
Window  ◇── imp ──>  WindowImp
  DrawText()           DevDrawText()  (soyut)
```

```
  DrawRect()           DevDrawLine()  (soyut)
   △── IconWindow       △── XWindowImp  {DevDrawText() -> XDrawString(),
   │     DrawBorder()   │                 DevDrawLine() -> XDrawLine()}
   │     (-> DrawRect(), DrawText())
   △── TransientWindow  △── PMWindowImp {DevDrawLine(), DevDrawText()}
         DrawCloseBox()
         (-> DrawRect())

DrawRect():  imp->DevDrawLine();  imp->DevDrawLine();
             imp->DevDrawLine();  imp->DevDrawLine();
```

`Window` ile `WindowImp` arasındaki `imp` ilişkisi birebir sahipliktir. GoF şemasında parça-bütün (elmas) olarak gösterilmiştir; hoca burada da farklı bir iş mantığı vurgulandığı için böyle çizildiğini, normal düz sahiplik okuyla gösterilmesinin de hiçbir sakıncası olmadığını söyledi.

Pencere soyutlaması bir dikdörtgen çizeceği zaman, günün sonunda dört çizgi çizmek gerekir; sol üst ve sağ alt köşelerin koordinatları bellidir. Ama çizginin nasıl çizileceği pencere yönetim sistemine göre değiştiği için `DrawRect()` metodu sahip olduğu gerçekleme nesnesine "şu koordinatlarla dört çizgi çiz" der. `WindowImp`'teki metotlar soyuttur; ondan kalıtımla X Window için, Presentation Manager için, GNOME ve KDE için ayrı gerçekleme sınıfları yazılır. Sol taraftaki pencere çeşitlemelerine de normal pencere, ikon pencere, şeffaf pencere gibi türler eklenebilir.

Artık yeni bir pencere yöneticisini desteklemek için yalnız bir gerçekleme sınıfı kardeş sınıf olarak eklenir (ör. `KDEWindowImp`); `IconWindow` ve `TransientWindow` yetenekleri onunla zaten çalışır. Yeni bir pencere türü gerektiğinde de pencere sınıfının somut alt sınıfları arasına bir kardeş eklemek yeter. Her ihtiyaç tek sınıf eklenerek, gerekenden fazla iş yapmadan karşılanır.

4. Bridge'in Yapısı ve Kullanım Durumları

4.1 Kalıp yapısı

Kalıp yapısı (ekrandan, 00:22:30):

```
client ──> Abstraction  ◇── imp ──>  Implementor
             Operation()               OperationImp()   (soyut)
               △                         △
     RefinedAbstraction        ConcreteImplementorA {OperationImp()}
                               ConcreteImplementorB {OperationImp()}

Operation():  imp->OperationImp();
```

Kalıp bileşenleri (slayttan):

- **Abstraction:** Uygulamanın temel işlemlerini oluşturan arayüzü belirler.
- **RefinedAbstraction:** Arayüzü özelleştirmeye yarar.
- **Implementor:** Uygulamanın iç yapısına yönelik arayüzü belirler.
- **ConcreteImplementor:** İç yapıyı gerçekleyen sınıf hiyerarşisidir.

Soyutlama da gerçekleme de soyut sınıflar üzerinden tanımlanır. Soyut soyutlama sınıfında işlemlerin, soyut gerçekleme sınıfının metotları üzerinden yapılması istenir. Böylece gerçeklemenin farklı somut alt sınıfları oluşturularak farklı ortamlarda çalışabilecek gerçeklemeler elde edilir; soyutlamanın farklı türevleri gerekiyorsa onlar da soyutlama sınıfından kalıtımla eklenir.

Hoca bir ayrıntının altını çizdi: `Implementor` soyut sınıfı aslında bir arayüz (interface) olarak da tanımlanabilir, ama `Abstraction` arayüz olarak tanımlanamaz. Çünkü soyutlama, özelleşmiş alt soyutlamaların mutlaka yapması gereken ortak işleri içinde barındırır ve belli bir operasyonu gerçeklemedeki metotlar üzerinden kodlaması gerekir. Bu yüzden abstraction tarafında interface kesinlikle kullanılamaz; implementor tarafında, mümkünse, kullanılabilir.

> **Ek bilgi (derste anlatılmadı):** Java'da bir `interface` örnek (instance) üye alanı taşıyamaz. `Abstraction`'ın gerçekleme nesnesini (`imp`) tutması ve kurucuda bu alana

değer ataması gerektiği için soyut sınıf olması zorunludur. `Implementor` ise yalnız metot imzalarından oluştuğu için arayüz olabilir.

4.2 Ne zaman kullanılır?

- Soyutlama ile gerçeklemeleri arasındaki bağın kalıcı olması istenmiyorsa; örneğin gerçekleme çalışma anında değiştirilebilmeli ise. Alternatif olan kalıtım hiyerarşisi çalışma anında değiştirilebilecek bir ilişki türü değildir; değişiklik için kodu değiştirmek gerekir.
- Örnekte olduğu gibi soyutlamaların ve gerçeklemelerin ayrı ayrı özelleşme ihtiyaçları varsa.

5. Bridge Uygulama Örneği: İki Çizim Kütüphanesi

5.1 Problem

Yine bir çizim programı üzerinde çalışılıyor. Şimdilik dikdörtgenler çiziliyor; ileride daire, üçgen, kare de çizilecek. Bir dikdörtgeni çizmek için sol üst köşenin `(x1, y1)` ve sağ alt köşenin `(x2, y2)` koordinatlarını bilmek yeter. Ama iki farklı "dikdörtgen hali" vardır. Hoca bunu bir vektör grafiğinin iki farklı dosya standardıyla kaydedilmesi olarak örnekledi: DP1 dosya biçiminde koordinat bileşenleri `x1, y1, x2, y2` sırasıyla, DP2 dosya biçiminde ise `x1, x2, y1, y2` sırasıyla verilmiştir.

Bir şeklin hangi biçimde olduğu nasıl ayırt edilir? Dosyanın başlığında (header) "bu birinci tür resim formatıdır" denip bütün şekiller öyle yorumlanabilir. Ya da daha zengin bir dosya biçiminde her `rectangle` etiketine `type=dp1` ya da `type=dp2` gibi bir parametre konur; o zaman aynı dosyada farklı ailelerden bileşenler bulunabilir. Her iki durumda da bir dikdörtgenin hangi aileden olduğu, dolayısıyla nasıl çizileceği bilinir.

Elimizde ayrıca DP1 ve DP2 adlı iki hazır çizim kod kütüphanesi vardır. DP1'de `draw_a_line()`, DP2'de `drawline()` metodu bulunur. Hoca bu kütüphanelerin statik metotlar kullandığını, bir nesne oluşturmaya gerek kalmadan sınıf kütüphanesi gibi düşünüldüğünü belirtti.

Hoca örneğe şöyle bir plan koydu: önce ilk akla gelen en basit çözüm görülecek, sonra bunu genişletmenin ne kadar fazla iş gerektirdiği görülecek, en sonda Bridge kalıbı uygulanıp nasıl bir esneklik kazandırdığı görülecek.

5.2 Önerilen çözüm (1)

Çizim programı client'tır. Dikdörtgenler bir sınıfla modellenir (ekrandan, 00:30:00):

```
Client ──> Rectangle (soyut)
             +draw()
             #drawLine()   (soyut)
              △
     V1Rectangle {#drawLine()} ··> DP1 {+draw_a_line()}
     V2Rectangle {#drawLine()} ··> DP2 {+drawline()}
```

Soyut `Rectangle` sınıfının somut bir `draw()` metodu vardır; bu metot dört kez protected ve soyut `drawLine()` metodunu çağırır. `drawLine` soyuttur, çünkü koordinat bileşenleri ailelere göre farklı sırada verilmiştir. `V1Rectangle`'ın `drawLine`'ı DP1'in `draw_a_line` metodunu, `V2Rectangle`'ınki DP2'nin `drawline` metodunu çağırır. Kod (ekrandan, 00:32:30):

```
package bridge.solution1;

public abstract class Rectangle {
    private double _x1, _x2, _y1, _y2;

    public void draw() {
        drawLine(_x1, _y1, _x2, _y1);
        drawLine(_x2, _y1, _x2, _y2);
        drawLine(_x2, _y2, _x1, _y2);
        drawLine(_x1, _y2, _x1, _y1);
    }
```

```
    abstract protected void drawLine(double x1, double y1,
                                     double x2, double y2);
}

public class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1,
                            double x2, double y2) {
        DP1.draw_a_line(x1, y1, x2, y2);
    }
}
```

Hoca üye alanların hangi sırada yazıldığının önemsiz olduğunu, önemli olanın çizim sırasında bileşenlerin hangi sırada verildiği olduğunu söyledi. `V2Rectangle` aynı yapıdadır; DP2'nin `drawline` metodunu çağırır ve koordinat bileşenlerini DP2'nin beklediği sırada verir.

5.3 Neden protected?

`drawLine` metodu nesnenin iç çalışmasıyla ilgilidir. Dikdörtgen kara kutu gibi kullanılmak istenir: ona yalnız "kendini çiz" denir; çizgi çizme işinin nasıl yapılacağına dışarıdan müdahale edilmek istenmez. Bu yüzden `drawLine` public olmamalıdır; normalde private olmalıdır. Ama private yapılsaydı kalıtımla `V1Rectangle` ve `V2Rectangle` sınıflarına aktarılamazdı. Bu yüzden üst sınıfta protected tanımlanır. Protected metotlar alt sınıflar için public gibi, diğer sınıflar için private gibi davranır. UML sınıf şemasında protected # ile gösterilir.

5.4 Daire eklenince: çözümün zayıf yönleri

Şimdi çember de eklenmek isteniyor ve onun da iki hali var: DP1'deki ve DP2'deki çember. Ayrıca çizim programı, ekrana boyarken "bu şekil dikdörtgense şu metodu, yuvarlaksa bu metodu çağır" gibi bir if-else zinciri yazmak istemez. Bunun yerine her şekle "nasıl çizersen çiz, kendini çiz" diyerek `draw` metodunu çağırmak ister; dikdörtgen kendini dikdörtgen olarak, çember çember olarak çizer. Üçgen, beşgen de eklenecektir. Bu yüzden dikdörtgen ve çember ortak bir `Shape` üst sınıfında birleştirilir; bütün şekiller tek bir `for` döngüsünde çizilebilir.

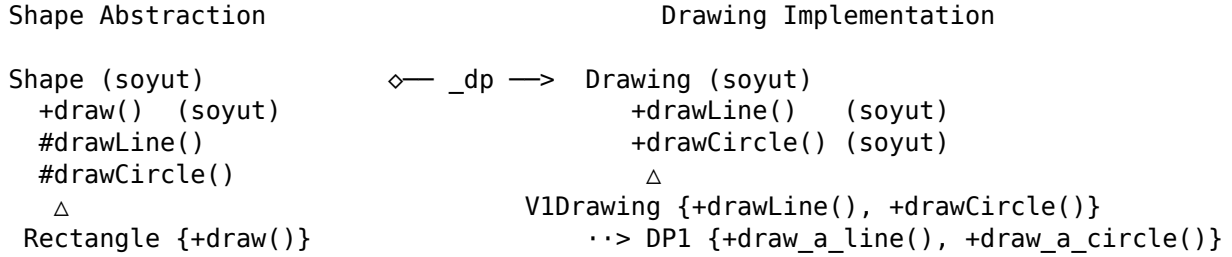
Önceki tasarım bozulmadan `Shape` (soyut `draw()`) eklenir; altında `Rectangle` ve `Circle` soyut sınıfları yer alır. `Circle`'ın somut `draw()` metodu protected `drawCircle()` metodunu çağırır; `V1Circle` ve `V2Circle` bu metodu sırasıyla DP1'in `draw_a_circle` ve DP2'nin `drawcircle` metotlarıyla gerçekler.

Bu tasarımın değerlendirmesi slaytta şöyle verildi (ekrandan, 00:36:30):

- Mevcut durumda 4 belirli şekil var (2 aile ve 2 şekil, $2 \times 2 = 4$).
- Yeni bir statik çizim metodu (DP3 kütüphanesi, üçüncü aile) eklemek gerekirse sınıf sayısı $3 \times 2 = 6$ olacak.
- Bunun üstüne yeni bir şekil (ör. üçgen) eklense sınıf sayısı $3 \times 3 = 9$ olacak.
- Her yeni eklenecek çizim metodu veya şekil türü ile sınıf sayısı fırlayacak.

5.5 Bridge kalıbıyla çözüm

Hoca problemi şöyle yorumladı: gereksinimlerde şekil türleri ile çizim yolları birbirinden ayrılmaz biçimde bir aradadır, ama kodlamada bunları bu kadar sıkı bağlamak zorunda değiliz. Şekil türlerini simgeleyen soyutlamalar ile farklı çizim ailelerini simgeleyen gerçeklemeler ayrılabilir. Bridge'in amacı da tam olarak budur. Çözüm (ekrandan, 00:40:00):



```
 Circle    {+draw()}              V2Drawing {+drawLine(), +drawCircle()}
                                      ··> DP2 {+drawline(), +drawcircle()}
```

Hoca şemayı anlatırken bir an `Shape`'teki `drawLine` ve `drawCircle` metotlarının da soyut olması gerekip gerekmediğini sorguladı, kalıp yapısına dönüp baktı ve kendini düzeltti: soyut olmayacaklar. Çünkü kalıbın yapısı gereği soyutlama, gerçeklemenin metotlarını çağırarak ilerler. `draw()` soyuttur; ama `drawLine` ve `drawCircle` somut ve protected'dır. Gövdeleri, `_dp` üye alanı üzerinden ilişkili olduğu `Drawing` alt sınıfı nesnesinin `drawLine` ya da `drawCircle` metodunu çağırmaktır. `_dp`, kalıptaki köprü nesnesidir.

Soyutlama tarafının kodu (ekrandan, 00:42:00):

```
package bridge.solution2;

public abstract class Shape {
    public abstract void draw();
    private Drawing _dp;
    public Shape(Drawing dp) { _dp = dp; }
    protected void drawLine(double x1, double y1,
                            double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x, double y,
                              double r) { _dp.drawCircle(x, y, r); }
}

public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle(Drawing dp,
                  double x, double y, double r) {
        super(dp);
        _x = x; _y = y; _r = r;
    }
    public void draw() { drawCircle(_x, _y, _r); }
}
```

`Shape`'in kurucusuna mutlaka bir gerçekleme (köprü) nesnesi verilmelidir. `Circle` orta noktası `(x, y)` ve yarıçapı `r` olan bir yuvarlaktır; kurucusu önce üst sınıfın kurucusuna gerçekleme nesnesini verir, sonra koordinat ve yarıçap alanlarını ilklendirir. `draw()` metodu üst sınıftan gelen `drawCircle` metodunu çağırır. Bu metotlar protected olduğu için alt sınıfta doğrudan çağrılabilir; private yapılsalardı buradaki çağrı derleme hatası verirdi. `Rectangle` de bir `Shape` alt sınıfıdır: iki köşesinin koordinatlarını tutar, kurucusunda önce bir çizim gerçeklemesiyle ilişkilendirilir, `draw()` metodunda üst sınıftan aldığı `drawLine` metodunu dört kez çağırır.

Gerçekleme tarafının kodu (ekrandan, 00:45:00):

```
package bridge.solution2;

public abstract class Drawing {
    public abstract void drawLine(double x1, double y1,
                                  double x2, double y2);
    public abstract void drawCircle(double x, double y,
                                    double r);
}

public class V1Drawing extends Drawing {
    public void drawCircle(double x, double y, double r) {
        DP1.draw_a_circle(x, y, r);
    }
    public void drawLine(double x1, double y1,
                         double x2, double y2) {
        DP1.draw_a_line(x1, y1, x2, y2);
    }
}
```

`Drawing`'in metotları soyuttur, çünkü nasıl kodlanacakları aileye (V1 mi, V2 mi) göre değişir. `V2Drawing` benzer biçimde DP2'nin metotlarını çağırır.

5.6 Kazanç ve çıkarılan ders

Üçgen eklenmek istenirse yalnız `Shape`'in alt sınıfı olarak, `Circle`'a kardeş bir `Triangle` sınıfı eklemek yeter. `Triangle` üç kez `drawLine` çağırır; kendisine hangi somut `Drawing` nesnesi verilmişse onun üzerinden DP1'in ya da DP2'nin çizgi metoduna ulaşır. Bazı makinelerde performansı daha iyi olan hazır bir DP3 kütüphanesi kullanılmak istenirse, `Drawing`'in alt sınıfı olarak bir `V3Drawing` eklenir. Yeni bir şekil için bir sınıf, yeni bir aile için bir sınıf: önceki tasarımda sınıf sayısı uçup gidiyordu, burada her ekleme tek sınıftır. Hoca en büyük kazancın bu olduğunu söyledi.

> **Hocanın vurgusu:** "Neyin değiştiğini bul ve onu sarmala" ilkesi. Varlıklar ve bu varlıkların ortaya çıkarılma (çizilme) biçimleri birbirinden farklı şekilde evrimleşir. Kendi içinde ortak olan ve başkalarından farklı biçimde evrimleşen şeyleri kendi kalıtım ağacına koymak en mantıklısıdır.

İlk kötü tasarımda, hele genişletilmiş hâlinde, hemen her şey kalıtımla yapılmıştı ve çok ağır bir kalıtım hiyerarşisi vardı. Daha iyi çözümde şekil ve çizimi ayrı hiyerarşilerde toplandı, aralarına bir sahiplik ilişkisi kondu. Kalıtım tamamen ortadan kalkmadı (öyle bir iddia da yok); ama bir kısım kalıtım ilişkisi sahiplik ilişkisine çevrildi ve daha esnek bir tasarım ortaya çıktı. Bu, "kalıtım yerine parça-bütün ilişkisini tercih et" ilkesinin bir uygulamasıdır.

6. Tartışma: Shape'in Sorumlulukları ve Adapter'lı Alternatif

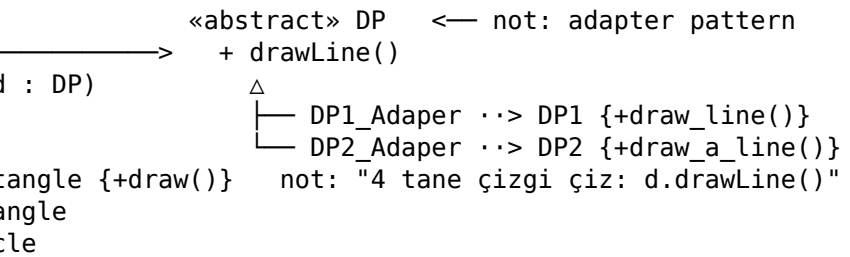
Aradan sonra bir öğrenci Bridge çözümü hakkında iki soru sordu. Hoca soruları Violet UML Editor'de şemalar çizerek tartıştı.

6.1 Shape'te drawLine ve drawCircle bulunmalı mı?

Soru: `Shape` sınıfı neden `drawLine` ve `drawCircle` metotlarını içeriyor? Bu, `Shape`'e fazla sorumluluk yüklüyor. `Shape`'te yalnız `draw` olsaydı, çizim işleri alt sınıflarda olsaydı daha iyi olmaz mıydı? Örneğin çizgiyle ya da daireyle çizilemeyen (integral işareti gibi) bir şekil eklendiğinde, `Shape`'e sürekli yeni metot eklemek gerekecek. Bunun yerine `Shape`'teki private `_dp` protected yapılsa, alt sınıflar doğrudan `_dp` üzerinden çizim yapsa?

Cevap: Hoca önce önerilen yapıyı çizdi (ekrandan, 01:10:00): `Shape` (`+draw()`), altında `Rectangle` (`+draw()`), `Triangle` ve `Circle`; `Rectangle.draw()` için "4 tane çizgi çiz" notu. Yanında DP1 (`+drawLine()`) ve DP2 (`+draw_a_line()`). Burada önemli bir nokta vardır: aslında tek bir `Rectangle` sınıfı yoktur; koordinatlar farklı sıralandığı için iki dikdörtgen türü vardır. Şekillerin `_dp` üzerinden ortak bir `drawLine` çağırabilmesi için DP1 ve DP2'nin ortak bir soyut `DP` üst sınıfından türemesi gerekir. Ama `DP1 extends DP` denemeyebilir, çünkü DP1 ve DP2'nin kaynak kodu elde olmayabilir. Bu durumda Adapter kullanılabilir.

Hoca alternatif tasarımı şöyle çizdi (ekrandan, 01:20:00):



Hoca adaptör kullanıldığını göstermek için önce kendi uydurduğu serbest bir gösterim («adapter» stereotipi) denedi, sonra bunun yerine UML not notasyonuyla "adapter pattern" notu düşmeyi daha doğru buldu. `Shape`'in kurucusuna DP1 adaptörü verilirse V1 tarzı, DP2 adaptörü verilirse V2 tarzı dikdörtgen elde edilir. Böylece problem Adapter kalıbıyla da çözülebilir.

Hoca iki çözümün sınıflarını saydı: Bridge çözümünde 8 sınıf (`Shape`, `Rectangle`, `Circle`, `Drawing`, `V1Drawing`, `V2Drawing`, DP1, DP2), Adapter'lı çözümde de 8 sınıf saydı (ekrandaki şemada

`Triangle` da bulunduğu için bu sayım büyük olasılıkla `Triangle` dışarıda bırakılarak yapıldı). Sınıf sayısı değişmedi.

Öğrenci, asıl kaygısının sınıf sayısı değil, on yeni şekil geldiğinde `Shape`'e sürekli metot yazmak olduğunu söyledi; her sınıf kendisinin nasıl çizileceğini bilmeli, `_dp` de protected olmalıdır. Hoca iki çözümde de dikdörtgenin kendisini dört `drawLine` çağrısıyla çizdiğini, birinde `Shape`'in, diğerinde `d`'nin `drawLine` metodunun kullanıldığını ve sonunda çağrının yine DP1'in ya da DP2'nin çizgi metoduna yönlendiğini, bu açıdan çok fark görmediğini söyledi. `Shape`'teki `drawLine`'da çizgi çizme kodu, yani iş mantığı yoktur; yalnız `_dp` üzerinden aktarma yapar.

> **Hocanın vurgusu:** Sorumlulukların doğru atanıp atanmadığını gerçekten ayrıntılı incelemek için her iki versiyon koda dökülür ve bağlaşım (coupling) ile uyum (cohesion) metriklerine bakılır; hangi tasarım daha iyi çıkarsa o daha doğrudur. Bu metrikleri hesaplamak için iskelet kodlar yetmez; hangi metodun içinde hangi üye alana erişildiği de bilinmelidir.

Gerçek bir projede bu aşamada karar vermek gerekirse, hoca objektif bir karar veremeyeceğini söyledi. Ancak daha önce benzer bir problem bir yolla çözülüp memnun kalınmadıysa diğer yol denenebilir; normalde bir yazılımcı "geçen sefer işe yaradı, ama farklı bir şey deneyelim" gibi bir maceraya pek atılmak istemez. Eğitimli bir karar, benzer bir şeyi iki yolla yapıp bağlaşım ve uyumu inceledikten sonra verilebilir. Öğrencinin tercihine itiraz etmedi: "olabilir, ama öbür türlü de olabilir".

Öğrenci ayrıca DP1 ve DP2 elde olsaydı Adapter'a hiç gerek kalmayacağını düşündüğünü söyledi; hoca DP1 ve DP2'nin kaynak kodunun elde olmadığının varsayıldığını hatırlattı.

7. Tartışma: Varlık Sınıflarına Davranış Yüklemek

Soru: Pattern'den bağımsız bir soru: `Shape`, `Rectangle`, üçgen, çember gibi sınıflar birer varlığı (entity) temsil ediyor. Bu sınıflara `draw` gibi davranışsal metotlar yüklemek uygun mu? Nesneye yönelik programlamada gerçek hayatı temsil etmeye çalışıyoruz; gerçek hayatta bu şekillerin kendini çizme yetkinliği yok. İnternette bazı kaynaklar entity sınıflarına davranış yazılabileceğini, bazıları yazılmaması gerektiğini söylüyor.

Cevap: Hoca, öbür türlü bir kod kusurunun (code smell) ortaya çıkacağını göstermek için yeni bir sınıf şeması çizdi (ekrandan, 01:30:00):

```
Drawer
  + drawRectangle( Rectangle )
  + drawCircle( Circle )
  + drawLine( ... )
     ··> Rectangle, Circle, Triangle   (parametre olarak alındığı için kullanma ilişkisi)

Rectangle
  - x1, y1, x2, y2 : int
  + getX1() : int
  + getX2() : int
  + getY1() : int
  + getY2() : int

not: public void drawRectangle( Rectangle r ) {
         drawLine( r.getX1(), r.getX2(), ...
     }
not: feature envy
not: long parameter list -> preserve Whole Object
```

Dikdörtgenin `draw` metodu yoksa, kendini nasıl çizeceğini bilmiyorsa, çizim işleri için bir `Drawer` sınıfı gerekir. Bu sınıfın `drawRectangle` metodu parametre olarak bir `Rectangle`, `drawCircle` metodu bir `Circle` alır. Dikdörtgenin `draw` metodu olmasa da `x1, y1, x2, y2` alanları ve bunların getter metotları olacaktır. `drawRectangle`'ın içinde ister istemez sürekli `r.getX1()`, `r.getX2()` gibi çağrılar yapılır ve sonuçlar `drawLine`'a parametre olarak verilir. `Drawer` dikdörtgenin, çemberin, şunun bunun koordinat verilerine sürekli erişmek zorunda kalır.

- **Feature envy:** Bir nesnenin sürekli başka bir nesnede tutulan verilere, hatta kendi üye alanlarından da çok başka nesnelerin üyelerine erişme ihtiyacı duymasıdır. Bu kusur ileride kod kusurları konusunda ayrıca görülecek. Tasarımda bir verinin en sık kullanıldığı yere konması ilkesi vardır; ama burada dikdörtgenin verisini `Drawer`'a koymak da mantıklı değildir.
- **Long parameter list:** `drawLine` hep bir nesneyi sorgulayıp `x1`, `x2` gibi değerleri parametre olarak alır ve parametre sayısı fazladır. Bunu çözmek için "preserve whole object" adlı bir yeniden düzenleme (refactoring) eylemi vardır. Hoca bu örnekte bunun anlamlı olmadığını belirtti: `drawRectangle` zaten tüm nesneyi parametre olarak alıyor; `drawLine`'ın parametrelerini ise kısaltmak mümkün değil.
- **Yalnız veri tutan nesne:** Davranış eklenmezse `Rectangle` bir veri nesnesi olur: alanları vardır ama basit getter/setter'dan başka metodu yoktur. Bir nesnenin yalnız veri depolama aracı olarak kullanılması pek mantıklı değildir.

Bu yüzden hoca ikinci yaklaşımı doğru bulmadığını söyledi; öğrenciden, ikinci yaklaşımın neden geçerli olduğuna dair kolay erişilebilir bir kaynak varsa paylaşmasını rica etti ki o tarafın argümanlarını da dinleyip daha eğitimli bir yargıda bulunabilsinler.

Öğrenci, model sınıfların veri depolamak için kullanılabildiğini, DTO (Data Transfer Object) ve DAO (Data Access Object) gibi yapıların çok kullanıldığını, dikdörtgen sınıfının amacının da onu tanımlamak, modellemek olduğunu söyledi. Hoca buna bir ilkeyle cevap verdi:

> **Hocanın vurgusu:** Bir metot (sorumluluk) birden fazla sınıfa konabiliyorsa, hangisine konacağına karar verirken şu düşünülür: ihtiyaç duyulan verinin en büyük kısmı hangi sınıfta yer alıyor? Metot o sınıfa konur. Böylece nesneler arasında en az veri aktarımı gerçekleşir.

Çizim için gereken temel veri koordinatlardır; o hâlde çizim sorumluluğunu koordinat verilerine sahip nesnenin içine koymak bu açıdan da mantıklıdır. Ancak farklı çizim yöntemleri ve aileleri olacağı için bunu Bridge ya da az önceki Adapter gibi bir tasarımla esnekleştirmek de gerekir. Hoca farklı alternatiflerin avantaj ve dezavantajlarının tartışılmasını güzel bir tartışma olarak değerlendirdi.

8. Composite Kalıbı: Amaç ve Örnek

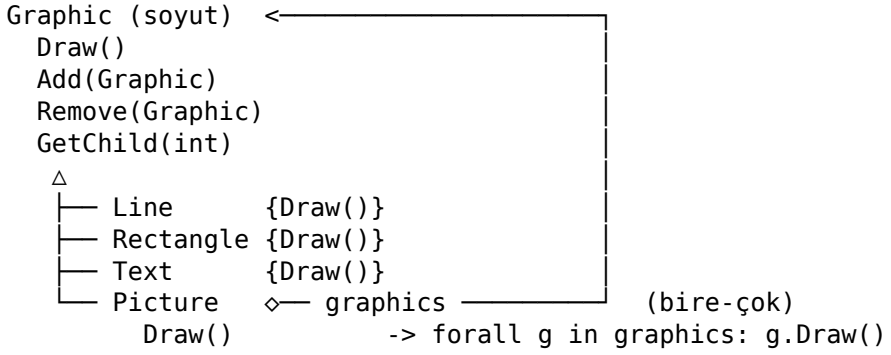
8.1 Amaç

Composite kalıbının amacı hiyerarşi biçimindeki parça-bütün ilişkilerini desteklemektir. Normalde parça-bütün ilişkisi tek düzeylidir: bir takım ve oyuncuları, bir ders ve onu alan öğrenciler, bir otobüs hattı ve duraklar gibi. Composite ise parçaların da kendi içinde parçalar içerebildiği ağaç biçimindeki ilişkiler içindir.

Benzer bir şey Word'de, PowerPoint'te çizim yaparken görülür: dört çizgi çekilir, birlikte seçilip gruplandırılır ve bu bir alt resim olur. Başka şeyler de kendi arasında gruplanıp ikinci bir alt resim olur. Sonra iki grup birlikte seçilip yeniden gruplanır; bu da daha üst düzey bir resimdir. Avantajı: bir grup, kendi içindeki düzen bozulmadan başka bir yere taşınabilir. Bu çok sık ihtiyaç duyulan bir yaklaşımdır.

8.2 Örnek çözüm

Örnek çözüm (ekrandan, 01:36:00):



```
Add(Graphic g)  -> add g to list of graphics
Remove(Graphic)
GetChild(int)
```

Çizgi, dikdörtgen ve metin sınıfları kendini çizebilir; zaten ortak bir üst sınıfları olacaktır. Bu ortak üst sınıf biraz daha yeteneklendirilir, çünkü işe "resim" kavramı da girer. Resmin normal bir çizim metodu olmalıdır; ek olarak `add` ile alt bileşen eklenebilir, `remove` ile bir bileşen silinebilir, `getChild` ile belli numaralı çocuk bileşen elde edilebilir. Bu metotlar üst sınıfta da soyut metotlar olarak yer almak zorundadır.

Parça-bütün ilişkisi `Graphic` türünden bir liste üzerinden saklanır: bir `Picture`, birden çok `Graphic` ile ilişkili olabilir. Her resim, içinde alt resimler ya da yaprak bileşenler barındırabilir. Yaprak, kendi içine daha fazla bileşen eklenemeyen bileşendir: bir çizgiye alt çizgiler eklenemez, çizgi başlı başına tek bir çizgidir. Bütün bileşenler (yapraklar ve resim) ortak `Graphic` soyut üst sınıfını gerçeklediği için, `Picture` tek bir `Graphic` listesi tutarak kendi türünden ya da herhangi bir yaprak türünden bileşenleri, ya da bunların herhangi bir karışımını içerebilir. Hoca burada bire-çok sahiplik olduğunu ve elmas kullanımının zorunlu olduğunu belirtti.

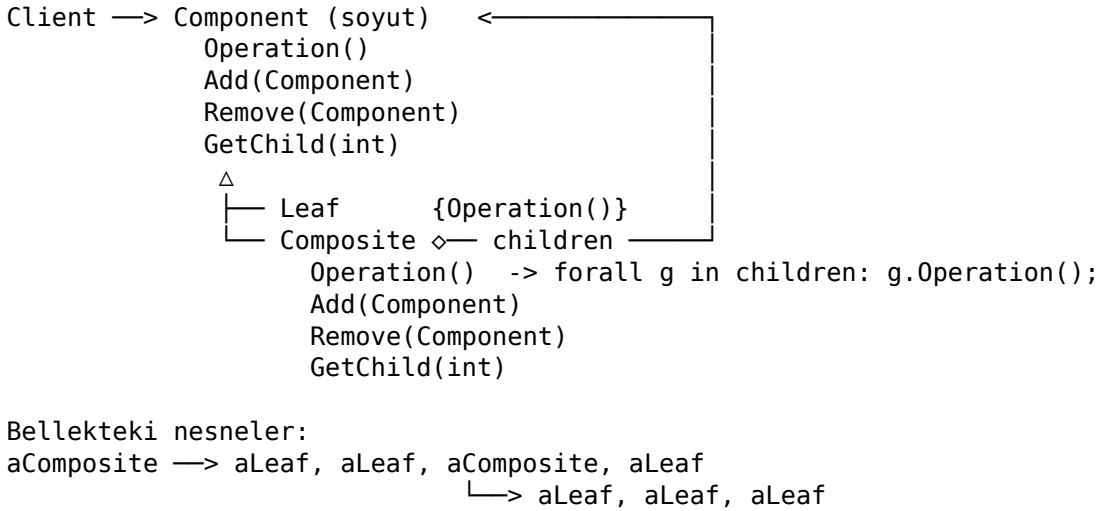
Bir resim `Draw` çağrısını şöyle işler: grafik listesindeki tüm nesnelere "kendini çiz" der. Bu nesneler çizgi, dikdörtgen ya da metinse kendilerini çizer; içlerinden biri başka bir resimse o da kendi içindeki tüm grafik nesnelerine çizme çağrısını iletir. Böylece bütün nesneler çizilmiş olur.

Hoca ağaç terminolojisini de hatırlattı: bir ağaç yapısında en altta yer alan ve daha fazla çocuğu olamayan düğümlere **yaprak**, hiyerarşinin en tepesine **kök**, geri kalan düğümlere **ara düğüm** denir.

9. Composite'in Yapısı ve Kullanım Durumları

9.1 Kalıp yapısı

Kalıp yapısı (ekrandan, 01:40:00):



Kalıp katılımcıları (slayttan):

- **Component:** Nesneler topluluğunun katılımcıları için ortak kullanım arayüzü sunar. Bir bütünün parçaları için erişim ve düzenleme metotları tanımlar.
- **Leaf:** Topluluktaki parçaları simgeler. Hiyerarşik yapının en altındaki düğümlerdir, çocukları olamaz.
- **Composite:** Hiyerarşik yapının ara düğümleridir. Farklı tür ara düğümler varsa bu sınıfın alt sınıfları şeklinde modellenir.
- **Client:** Parça ve toplulukları benzer şekilde kullanabilir. Client bir Visitor olabilir.

`Component`'in dört metodu da soyut olduğu için bu sınıf aslında bir arayüz (interface) olarak da tanımlanabilir. `Add`, `Remove` ve `GetChild`, parça-bütün ya da hiyerarşik ilişkinin getirdiği metotlardır. Bunların dışındaki operasyonlar probleme göre değişir: bu bir resimse çizim, bir

makine ve parçalarıysa toplam maliyetin ya da üretim süresinin hesaplanması gibi farklı raporlamalar olabilir. Çok sayıda farklı operasyon olabilir, ama hepsinin ana mantığı aynıdır: “her bileşen için ilgili operasyonu yap”. Bellekte ise hiyerarşinin kökünde bir composite nesnesi; onun yaprak çocukları ya da alt composite çocukları; alt composite’lerin de kendi yaprak ya da composite çocukları bulunur.

Hoca, slayttaki “Client bir Visitor olabilir” maddesine ilişkin kalıbın henüz görülmediğini, ileride işleneceğini söyledi.

9.2 Ne zaman kullanılır?

- Çeşitli varlıklar arasında hiyerarşik bir parça-bütün ilişkisi varsa.
- Bir nesne topluluğundaki bireysel nesnelere ortak bir erişim tarzı gerekiyorsa.

Bunlar GoF’un kalıbın kullanılabileceğini söylediği temel durumlardır.

10. Composite Gerçekleme Ayrıntıları ve Zayıf Yönleri

10.1 Çocuk ebeveynini bilmeli mi?

İlişki genelde üst bileşenden alt bileşene, yani ebeveynden çocuğa doğru gider. Çocuğun da ebeveynini bilmesi gerekir mi? Gerekiyorsa alt düğüm üst düğümden haberdar edilir, gerekmiyorsa edilmez. Alternatif olarak bir alt düğüm yalnız üst düğümünden, ya da üst düğümünü atlayıp hiyerarşinin kökünden, ya da ikisinden birden haberdar olabilir. Bu haberdarlıkların sayısı arttıkça yapılması gereken tutarlılık kontrolleri de ağırlaşır. Gerekirse yapılır, gerekmiyorsa yapılmaz.

10.2 Bir düğümü kim, nasıl siler?

Hoca bunun çok önemli bir soru olduğunu söyledi. Bir alternatif, hiyerarşinin köküne “şu düğümü sil” demektir. Diğeri, bir bileşene (yaprak ya da alt composite olabilir) “kendini bulunduğun yerden sil” demektir. Ya da herhangi bir nesneye “senin hiyerarşinin içinde şöyle bir composite varsa git onu sil” denebilir. Bu ikinci tür yaklaşımlarda düğümün üst düğümünden ya da hiyerarşinin kökünden haberdar olması zorunludur; başka bir yol yoktur.

Bir ara düğüm silindiğinde ne olacağı da ayrı bir karardır: örneğin bir composite silinmek istenirse, onun çocuğu olan yapraklar da mı silinecek, yoksa bir üst ebeveyne mi atanacak? Bu kararlar, çözülmek istenen gerçek dünya problemi neyi gerektiriyorsa ona göre verilir. Bunların önceden bilinmesi gerekir; gereksinimlerin en başta belirlenmesi bu yüzden önemlidir.

10.3 Hangi veri yapısı?

Farklı ağaç veri yapıları vardır: ikili ağaç (binary tree), kırmızı-siyah ağaç (red-black tree) ve başka çeşitli ağaç yapıları. Literatürde tek bir tür ağaç yapısı yoktur; hangisinin kullanılacağı da önemli bir sorudur.

10.4 Çocuk listesi üst sınıfa taşınabilir mi?

Normalde çocukları yalnız composite içereceği için çocuk listesi composite alt sınıfında tutulur. Bu liste belki `Component` üst sınıfında da tutulabilir. Ama öyle yapılırsa, birincisi `Component`’i arayüz yapabilme esnekliğinden feragat edilmiş olur; ikincisi kalıbın aşağıda anlatılan zayıflığı belirginleşir.

10.5 Kalıbın zayıflığı: kalıtımın özüne aykırılık

> **Hocanın vurgusu:** Bu kalıp aslında kalıtım ilişkisinin özüne aykırı bir davranışta bulunur. Üst sınıftaki bazı metotların bazı alt sınıflar için anlamı yoktur.

`Component`’in `add`, `remove` ve `getChild` metotları composite için anlamlıdır, leaf için hiçbir anlamı yoktur: yaprağa bileşen eklenemez, ondan bileşen silinemez, çocuğu alınamaz. Ama kalıtımda “ben şu metotları almıyorum” denemez. Yapılabilecek tek şey, `add` ve `remove` metotlarını

leaf sınıfında boş gövdeli tanımlamaktır (hoca bunun hiç hoşuna gitmediğini söyledi). `getChild` ise boş bırakılamaz, bir şey döndürmesi gerekir; örneğin:

```
public Component getChild(int childID) {
    return null;
}
```

Kalıtımın özü alt sınıfların üst sınıfın daha özelleşmiş hâli olmasıdır; leaf ile component arasındaki durum bunun tersidir. Bu, kalıp için önemli bir eleştiridir. Yine de hiyerarşik parça-bütün ilişkilerini simgelemekteki kullanışlılığı yüzünden tercih edilebilir.

Bir başka olası dezavantaj: hoca diğer kalıplarda bu kadar çok gerçekleme ayrıntısından söz etmediklerini, bu kalıbı kullanırken programcının vermesi gereken karar sayısının diğer kalıplara göre çok fazla olduğunu söyledi. Bu kararların bazıları kullanılan ağaç veri yapısında zaten çözülmüş olabilir; o zaman bu bir dezavantaj oluşturmaz. Ama kalıtımın özüne aykırı davranmanın pek bir mazereti yoktur.

11. Uygulama Örneği: Makine Üretim Kataloğu

11.1 Problem

Hoca bu örneği Design Patterns Java Workbook kitabından birebir almadığını, ondan esinlenerek değiştirdiğini söyledi; kitaptaki problem küçük değişikliklerle farklı kalıplara yönelik güzel örneklere dönüşebilen bir problemdir.

Bir şirketin makine ve makine parçası üretim kataloğu vardır. Birden fazla fabrikada çeşitli makineler ve makine parçaları üretilir. Yayınlanacak katalogda ürünlerin hangi fabrikada üretildiği, maliyeti gibi bilgiler bulunur ve bu bilgilerden farklı raporlar üretilmesi istenir. Bazı makineler kendi içinde alt makineler ve makine parçaları içerebilir. Problem alanının bileşenleri hiyerarşik bir parça-bütün ilişkisi gösterdiği için ilk akla gelen kalıp Composite'tir. Sıfırdan bir tasarım yapıldığı, önceden bir çözüm ya da veri yapısı olmadığı için hoca doğrudan Composite ile çözüme geçti.

11.2 Tasarım

Tasarım (ekrandan, 02:10:00):

```
«abstract» ManufacturingItem
  - machineID : int
  - name, factory : String
  + ManufacturingItem( machineID : int, name : String, factory : String )
  + getMachineID( ) : int
  + getName( ) : String
  + getFactory( ) : String
  + setFactory( String )
  + getParentID( ) : int { abstract }
  + setParentID( int ) { abstract }
  + getParts( ) : Iterator<ManufacturingItem> { abstract }
  + addPart( ManufacturingItem part ) : boolean { abstract }
  + removePart( ManufacturingItem part ) : boolean { abstract }

Machine  ──▷ ManufacturingItem;   Machine ◇──> ManufacturingItem
  - parts : ArrayList<ManufacturingItem>
  + Machine( machineID : int, name : String, factory : String )
  + getParentID( ) : int
  + setParentID( int )
  + getParts( ) : Iterator<ManufacturingItem>
  + addPart( ManufacturingItem part ) : boolean
  + removePart( ManufacturingItem part ) : boolean

MachinePart  ──▷ ManufacturingItem
  + MachinePart( machineID : int, parentID : int, name : String, factory : String )
```

```
  + getParentID( ) : int,  + setParentID( int )
  + getParts( ) : Iterator<ManufacturingItem>
  + addPart( ManufacturingItem part ) : boolean
  + removePart( ManufacturingItem part ) : boolean

Report  1 ··> * ManufacturingItem
MainApp ··> Report
```

Makinelerin ve makine parçalarının birer kimliği (ID), adı ve üretildiği fabrika vardır. Bu üç bilgi her ikisinde de bulunduğu için, Composite kullanılmasa bile bunları tutan soyut bir üst sınıf oluşturmak zaten mantıklı bir modellemedir. Aradaki ağaç yapısını modellemek için Composite uygulanır: composite (ara düğüm) bileşen makine, yaprak düğüm makine parçasıdır. Bu yüzden `Machine` sınıfında bir `parts` listesi vardır; hoca rahat olduğu için generic `ArrayList` kullandı.

Parçadan bağımsız işlemler üst sınıfta somut olarak kodlanır: zorunlu üye alanları ilklendiren kurucu, getter metotları ve bir parçanın fabrikası değişebileceği için `setFactory`. Bir alt bileşenin üst ebeveynini öğrenmek için `getParentID` vardır. Makinenin alt parçaları üzerinde dolaşmak için bir iteratör döndüren `getParts`, parça eklemek için `addPart`, parça silmek için `removePart` vardır; son ikisi işlem başarılıysa `true`, değilse `false` döndürür. Kurucu ve getter/setter dışındaki metotlar (`parentID` ile ilgililer dahil) soyuttur; gövdeleri ancak alt sınıflarda verilebilir.

Hoca şemaya iki düzeltme ekledi. Birincisi, makinenin bir ebeveyni olmaz, çünkü o kök bileşendir; ama `MachinePart`'ın bir `parentID` üye alanı olması gerekir ve şemada bu eksiktir. İkincisi, fabrika `String` olarak modellenmiştir; en doğrusu bir `Factory` sınıfı olmasıdır (o fabrikada üretilen nesneleri modellemek için, kendi içinde `ManufacturingItem`'lar içeren ayrı bir sınıf). Hoca zaten karmaşık olan örneği daha fazla karmaşıklaştırmamak için bu hızlı çözümü seçtiğini söyledi.

Bir parça birden fazla makinede yer alırsa? Bir makine parçası için tek bir nesne oluşturarak bu çözülemez, çünkü her parçanın bir `parentID`'si vardır. Bu örnekte aynı parçanın, yalnız `parentID`'leri farklı, diğer üye alanları aynı kopyaları oluşturulur. İlla tek bir parça için tek bir nesne olmalı denirse, nesne kimliğine önem veriliyor demektir; o zaman makine ile makine parçası arasındaki ilişkiyi kuracak ekstra bir sınıf gerekir ve bu da örneği karmaşıklaştırır. Hoca kopya oluşturmanın bir sakıncası olmadığı varsayımıyla ilerledi.

11.3 Machine ve MachinePart kodu

`ManufacturingItem`, UML şemasından çıkarılabileceği gibi üç üye alanı, bunları alan kurucuyu ve getter/setter metotlarını normal kodlar; `parentID` dahil diğer metotlar soyuttur.

`Machine` sınıfı (ekrandan, 02:20:00):

```
public Machine( int machineID, String name, String factory ) {
    super(machineID, name, factory);
    parts = new ArrayList<ManufacturingItem>();
}

public int getParentID() { return 0; }
public void setParentID(int parentID) { }

public boolean addPart( ManufacturingItem part ) {
    part.setParentID(getMachineID());
    for( ManufacturingItem aPart : parts )
        if( aPart == part )
            return false;
    return parts.add(part);
}
public boolean removePart( ManufacturingItem part ) { return parts.remove(part); }
public Iterator<ManufacturingItem> getParts( ) { return parts.iterator(); }
```

Makinenin ebeveyni olamayacağı için `getParentID` doğrudan 0 döndürür, `setParentID`'nin gövdesi boştur. Hoca, gelen değeri dikkate almadan sıfıra eşitlemek de mümkün olsa, hep sıfır kalacak bir bilgi için ekstra bir üye alan tanımlama zahmetine gerek olmadığını söyledi. Üst sınıftan gelen metotlar mecburen kodlanır; bu, kalıtımın özüne aykırıdır, ama bu eleştiri kalıp anlatılırken zaten yapılmıştı.

`addPart` önce eklenen parçaya "artık sen benimsin, senin üst makinen benim" der ve `setParentID(getMachineID())` çağırır. Sonra mevcut parçalar arasında o parçayı arar; zaten varsa yeniden eklemez ve `false` döner. Yoksa parçayı listeye ekler ve listenin `add` metodunun döndürdüğü sonucu döndürür (bellek kalmaması gibi istisnai durumlar dışında eklenememesi için neden yoktur). Hoca, 17. satırdaki `setParentID` çağrısını döngüden sonraya almayı düşündüğünü, ama parça zaten bu makineye aitse ebeveyni zaten bu makine olacağı için yerinin fark etmediğini söyledi. `removePart`, `ArrayList`'in (ya da `LinkedList`'in, fark etmez) kendi `remove` metodunu kullanır. `getParts`, alt parçalar ve alt makineler arasında rahatça dolaşılabilmesi için listenin iteratörünü döndürür.

`MachinePart` sınıfında `parentID` tanımlı olmak zorundadır. Kurucu ilk bilgileri `ManufacturingItem`'a aktarır ve kendi `parentID`'sini atar. `parentID` için getter ve setter vardır, çünkü bir parçayı bir makineden başka bir makineye taşımak gerekebilir. Parçanın alt parçaları olmadığı için `getParts` `null` döndürür; parçaya ek parça eklenemeyeceği için ekleme metodu doğrudan `false` döndürür; `removePart` da benzer biçimde `false` döndürür.

11.4 Report ve çalıştırma

Rapor sınıfı belirli bir fabrikada üretilen öğeleri listeler (ekrandan, 02:23:30 ve 02:34:00):

```
package dp.composite.solution1;

import java.util.*;

public class Report {
    public static int findItems( ManufacturingItem items[], String factory ) {
        int nr = 0;
        System.out.println("Items manufactured in " + factory + ":");
        System.out.println("Nr.\tID\tName");
        System.out.println("-----------------------------------");
        for( ManufacturingItem item : items ) {
            if( item==null )                continue;
            if( item.getFactory().compareToIgnoreCase(factory) == 0 ) {
                nr++;
                System.out.println( nr + ".\t" + item.getMachineID() +
                        "\t" + item.getName() );
            }
            Iterator<ManufacturingItem> subItems = item.getParts();
            if( subItems==null )            continue;
            while( subItems.hasNext() ) {
                ManufacturingItem part = subItems.next();
                if( part.getFactory().compareToIgnoreCase(factory) == 0 ) {
                    nr++;
                    System.out.println( nr + ".\t" + part.getMachineID() +
                            "\t" + part.getName() );
                }
            }
        }
        ...
    }
}
```

Metodun sonu ekranda görünmedi; çıktıdaki "3 parça bulundu" satırını yazdırdığı ve büyük olasılıkla `nr` değerini döndürdüğü anlaşılıyor. `findItems` statik tanımlanmıştır, çünkü bir öğe dizisi içinden belli bir fabrikada üretilenleri süzen bir raporlama fonksiyonudur. Önce başlık, numara, sekme, ID, sekme ve ad sütunlarını yazar. Ardından `for each` döngüsüyle her öğenin

fabrika adını aranan adla karşılaştırır. Hoca büyük-küçük harf duyarlı olmasını istemediği için `compareToIgnoreCase` kullandı: "ANKARA" da "ankara" da kabul edilir. Eşleşme varsa sayaç bir artırılır ve öğenin ID'si ile adı yazdırılır.

Öğe bir makine de olabilir, makine parçası da. Makineyse alt parçaları da araştırılmalıdır: makinenin kendisi aranan fabrikada üretilmiyor olabilir, ama alt parçalarından biri üretiliyor olabilir. Bu yüzden `getParts()` ile alt öğelerin iteratörü alınır ve eleman olduğu sürece dolaşılır; her biri için yine aynı fabrika kontrolü yapılır.

Ana program (ekrandan, 02:30:00):

```
package dp.composite.solution1;

public class MainApp {
    public static void main(String[] args) {
        ManufacturingItem[ ] items = new ManufacturingItem[4];
        items[0] = new MachinePart(99, 127, "Krank şaftı KŞ-7", "Bursa");
        Machine jenerator = new Machine(135, "Jeneratör JMX-99", "İstanbul" );
        jenerator.addPart( new MachinePart(259, 135, "Flanş FTR-5", "Bursa") );
        jenerator.addPart( new MachinePart(378, 135, "Kasnak KS-9", "İstanbul") );
        jenerator.addPart( new MachinePart(196, 135, "Rulman RL-3", "Ankara") );
        items[1] = jenerator;
        items[2] = new MachinePart(63, 76, "Distribütör DST-12", "Bursa");
        items[3] = new MachinePart(82, 19, "Pnömatik Vana VPN-7", "Ankara");
        ...
    }
}
```

`MachinePart` kurucusunun parametreleri sırasıyla makine ID'si, ebeveyn ID'si, ad ve fabrikadır. İlk öğe 99 ID'li, 127 ID'li bir makineye ait, Bursa'da üretilen bir krank şaftıdır. İkinci öğe 135 ID'li jeneratör makinesidir ve içine üç parça eklenmiştir. Üçüncü ve dördüncü öğeler yine makine parçalarıdır. Ardından raporun Bursa için çalıştırılması istenir. Hoca, beklenen sonucun 7., 9. ve 13. satırlarda oluşturulan üç öğe olduğunu söyledi.

Çıktı (ekrandan, 02:30:00):

```
Items manufactured in Bursa:
Nr.     ID      Name
---------------------------------
1.      99      Krank şaftı KŞ-7
2.      259     Flanş FTR-5
3.      63      Distribütör DST-12
        3 parça bulundu.
```

Jeneratörün kendisi İstanbul'da üretilir, ama parçalarından flanş Bursa'da üretildiği için listelenir. Son makine parçası olan distribütör de Bursa'dadır.

> **Ders sonrası not:** Hoca null denetimlerinin gerekçesini, bir koleksiyonda dolaşırken arada null bileşenler gelebilmesi olarak açıkladı. Bu programda `items` dizisinin dört elemanı da doludur; asıl gerekli olan denetim büyük olasılıkla `subItems == null` denetimidir, çünkü `MachinePart.getParts()` `null` döndürür ve denetim olmasa `subItems.hasNext()` çağrısı `NullPointerException` verirdi. `item == null` denetimi ise dizide boş eleman kalması olasılığına karşı bir güvenliktir.

Hoca örnekten şu dersleri çıkardı: tasarım kalıbının zayıflığı olan, pek anlamı olmayan metotlar örnekte somut olarak görüldü; kullanılan veri yapısının gerçeklemesine yönelik bir ayrıntı (`getParts`'ın `null` dönmesi) yüzünden null kontrolü yapmak gerekti, yani kullanılan veri yapısının ve hatta onun gerçeklendiği platformun özel noktalarını bilmek gerekir; bir makine parçasının birden fazla makineye ait olabileceği durumda tek kopya isteniyorsa daha fazla kodlama gerekeceği, birden fazla kopya oluşturulabiliyorsa ise işin daha rahat kodlanabileceği görüldü.

> **Ek bilgi (derste anlatılmadı):** `Report.findItems` yalnız bir düzey aşağı iner: bir makinenin doğrudan parçalarına bakar, ama alt makinelerin içindeki parçalara bakmaz. Composite'in tam ağaç yapısında (makine içinde alt makine) bütün düğümleri

gezmek için tarama işleminin özyinelemeli (recursive) yazılması ya da işlemin `Operation()` gibi bileşenlerin kendisine konup composite'in onu çocuklarına iletmesi gerekir. `Machine.addPart`'taki `aPart == part` karşılaştırması da içerik eşitliğine değil nesne kimliğine (referans eşitliğine) bakar.

12. Kod Kusuru: Tekrarlanan Kod ve ReportV2

Hoca, `Report` ile `ReportV2` arasındaki farkı çalıştırarak gösterdi; iki rapor aynı çıktıyı üretir. Asıl göstermek istediği bir kod kusuruydu: tekrarlanan kod (duplicate code). Kod kusurları konusu ilerleyen haftalarda ayrıca işlenecek.

`Report.findItems`'ta hem ana öğeyi hem alt parçayı ekrana yazdırmak için neredeyse aynı satırlar iki kez yazılmıştır: 12–17. satırlardaki kodun hemen hemen aynısı 22–26. satırlarda da vardır. Yeni bir sütun eklemek gibi bir değişiklik iki yerde yapılmak zorunda kalır; birinde hata varsa onu düzeltince öbürünü de düzeltmek gerekir. Bunun daha bilinen adı kopyala-yapıştır (copy-paste) problemidir; kod kusuru terminolojisinde "duplicate code" denir.

`ReportV2`'de kod toparlandı: bir öğeyi (makine ya da makine parçası, fark etmez) ekranda göstermek için `showItem` adlı bir metot yazıldı ve yazdırma satırı tek bir yerde toplandı. Her öğe önce ekrana yazdırılır, sonra alt öğeleri alınır; alt öğesi yoksa bir sonraki döngü adımına geçilir, varsa alt öğeler de sırayla listelenir. Bunun bir bedeli oldu: önceki hâlde `findItems` içinde yerel değişken olan, kaçıncı sırada listeleneceğini tutan sayaç, ikinci versiyonda üye alan olarak tanımlanmak zorunda kaldı; aksi hâlde metotların daha fazla parametre alması gerekirdi. Hoca bunu daha derli toplu bulduğunu söyledi ve ilk rapor koduna "duplicate code" notu düştü. ReportV2'nin kodu bu notta incelenen karelerde yer almadı.

Örnek Sorular

Soru 1: Sınıf sayısı ne kadar büyür? (ekrandan, 00:36:30)

Dikdörtgen ve daire şekilleri, DP1 ve DP2 çizim kütüphaneleriyle tek kalıtım hiyerarşisinde (`V1Rectangle`, `V2Rectangle`, `V1Circle`, `V2Circle`) modellenmiştir. a) Mevcut durumda kaç belirli şekil sınıfı vardır? b) Yeni bir çizim kütüphanesi (DP3) eklenirse? c) Bunun üstüne üçgen de eklenirse? d) Aynı eklemeler Bridge çözümünde kaç yeni sınıf gerektirir?

Derste verilen cevap: a) 2 aile × 2 şekil, $2 \times 2 = 4$. b) $3 \times 2 = 6$. c) $3 \times 3 = 9$. d) Bridge çözümünde DP3 için yalnız `V3Drawing`, üçgen için yalnız `Triangle` eklenir; her ekleme tek sınıftır.

Soru 2: Başka uyumsuz sınıflar için adaptör (derste sözlü, 00:12:30)

Çizim programına `XXCircle` bir adaptörle bağlandı. İleride daha önce kodlanmış `XXPentagon` gibi başka bir uyumsuz sınıf da eklenecekse ne yapılmalı?

Derste verilen cevap: Her uyumsuz sınıf için ayrı bir adaptör sınıfı yazılır; örneğin `Shape`'in alt sınıfı olarak, metotlarını `XXPentagon`'un karşılıklarına ileten bir `Pentagon` sınıfı.

Soru 3: Bridge örneği Adapter ile çözülebilir mi? (derste sözlü, 01:05:20)

Bridge örneğinde `Shape`'in `drawLine`/`drawCircle` metotlarını taşıması yerine her şeklin kendini doğrudan bir çizim nesnesi üzerinden çizmesi istenirse ve DP1, DP2'nin kaynak kodu elde değilse nasıl bir tasarım kurulur? İki tasarımdan hangisi daha iyidir?

Derste verilen cevap: Soyut bir `DP` sınıfı (`drawLine()`) ve DP1, DP2 için `DP1_Adaper`, `DP2_Adaper` adaptörleri kurulur; `Shape(d: DP)` kurucusu bir adaptör alır ve `Rectangle.draw()` dört kez `d.drawLine()` çağırır (Bölüm 6). İki tasarımda da 8 sınıf vardır. Hangisinin daha iyi olduğu, kodlanmış hâllerinin bağlaşım ve uyum metrikleri karşılaştırılarak ya da benzer bir problemdeki deneyimle belirlenir; iskelet kodla önceden objektif bir karar verilemez.

Soru 4: Varlık sınıfları kendini çizmeli mi? (derste sözlü, 01:22:30)

`Rectangle` gibi bir varlık sınıfına `draw` davranışı yüklemek yerine çizim ayrı bir `Drawer` sınıfına verilirse hangi sorunlar doğar?

Derste verilen cevap: `Drawer.drawRectangle(Rectangle r)` sürekli `r.getX1()` gibi başka nesnenin verisine erişir (feature envy), çizim metotlarının parametre listesi uzar (long parameter list) ve `Rectangle` yalnız getter'ları olan bir veri nesnesine dönüşür. Sorumluluk, gereken verinin en çoğuna sahip sınıfa, burada koordinatları tutan şekle verilmelidir (Bölüm 7).

Soru 5: Bursa'da üretilen öğeler (ekrandan, 02:30:00)

Bölüm 11.4'teki `MainApp` için `Report.findItems(items, "Bursa")` çalıştırılırsa hangi öğeler listelenir?

Derste verilen cevap (program çıktısı): 1. `99 Krank şaftı KŞ-7`, 2. `259 Flanş FTR-5`, 3. `63 Distribütör DST-12`; "3 parça bulundu". Jeneratör (İstanbul) listelenmez, ama onun Bursa'da üretilen parçası flanş listelenir. Kasnak (İstanbul), rulman (Ankara) ve pnömatik vana (Ankara) listelenmez. Sonuç python ile de doğrulandı.

Dersin Toparlaması

Bu derste yapısal kalıplardan üçü işlendi. Adapter, yalnız metot adları uyumsuz olan hazır bir sınıfı, aradaki bir sınıfın çağrıları ona iletmesiyle mevcut tasarıma bağlar. Bridge, bir soyutlamayı gerçeklemesinden ayırarak iki bağımsız değişim eksenini iki ayrı kalıtım hiyerarşisine koyar; aralarındaki sahiplik ilişkisi köprüdür. Bu sayede şekil ve çizim kütüphanesi örneğinde çarpımsal büyüyen sınıf sayısı, her ekleme için tek sınıfa iner. Bridge örneği üzerine yapılan tartışmada aynı problemin Adapter ile de çözülebileceği görüldü; hangi tasarımın daha iyi olduğunun ancak bağlaşım ve uyum ölçülerek söylenebileceği, sorumluluğun da gereken veriye en çok sahip sınıfa verilmesi gerektiği vurgulandı. Feature envy, long parameter list ve duplicate code gibi kod kusurlarına ilk kez değinildi.

Composite, ağaç biçimindeki parça-bütün ilişkilerini tek bir ortak üst tür üzerinden modeller; istemci tek nesneleri ve nesne gruplarını aynı biçimde kullanır. Buna karşılık yaprakların anlamsız `add/remove/getChild` metotlarını taşıması kalıtımın özüne aykırıdır ve kalıp, ebeveyn bilgisi, silme ve veri yapısı gibi pek çok gerçekleme kararı gerektirir. Makine üretim kataloğu örneği bu güçlü ve zayıf yanları kod üzerinde gösterdi. Hoca, yapısal kalıpların kalanının (Facade dahil) bir sonraki derste tamamlanmasını, ardından davranışsal (behavioral) kalıplara geçilmesini planladığını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.